Tiered Memory구조를 위한 LSM Tree구조 개선 연구

이승윤[0], 김상훈
아주대학교 소프트웨어학과

soccer1356@ajou.ac.kr, sanghoonkim@ajou.ac.kr

LSM Tree Improvement for Tiered Memory

Seungyun Lee[0], Sang-Hoon Kim
Department of Software and Computer Engineering, Ajou University

요 약

이 연구는 Tiered Memory 구조에서 사용 가능한 LSM Tree 구조에 대하여 연구를 진행한다. 주요 내용은 로그 파일에서 데이터를 관리하고, 가비지 컬렉션 중에 빈도가 낮은 데이터를 Slow Memory로 이동시키면서, 자주 사용하는 데이터는 Fast Memory에 유지하는 새로운 접근법을 소개한다. 이를 통해 File System보다 빠른 Slow Memory를 사용하여 개선된 I/O를 목표로 한다.

1. 서 론[1)]

최근 컴퓨터 하드웨어 간의 데이터 전송 속도를 크게 향상시키는 CXL 기술이 개발되었다. CXL(Compute Express Link) 인터페이스는 메인 메모리의 확장을 가능하게 하고 Tiered Memory 시스템을 제공한다. 이 연구에서는 데이터를 저장하고 관리하는 한 방식인 LSM Tree를 Tiered Memory 구조에 맞게 개선하는 방법을 모색한다. LSM Tree는 주로 Key-Value 형태의 데이터를 저장하는 데 사용된다. 기본적으로, LSM Tree는 Monolithic구조로 이루어져 있다. 따라서, Tiered Memory구조에서는 효율적이지 않다. 본 연구는 Tiered Memory 구조에서 어떻게 LSM Tree를 최적화할 수 있는지를 탐구한다.

2. 관련 연구

2.1 Tiered Memory

Tiered Memory는 여러 종류의 메모리 기술을 계층적으로 구성하는 개념이다. 빠른 속도의 상위 계층과 더 느리지만 용량이 큰 하위 계층을 함께 사용하는 것이 대표적인 Tiered Memory 구조이다.

이 구조에서 핵심 아이디어는 데이터의 특성에 따라 적절한 메모리 계층을 선택하는 것이다. 자주 사용되는 데이터는 빠른 상위 계층에 저장하고, 그렇지 않은 데이터는 용량이 큰 하위 계층에 저장한다. 이렇게 데이터를 효율적으로 분산 배치함으로써 전체적인 시스템 성능을 높일 수 있다.

2.2 Key-Value Store

Key-Value Store는 데이터를 Key와 Value의 쌍으로 저장하는 비 관계형 데이터베이스의 한 형태이다. 이 구조는 단순하지만, 그 단순함이 곧 높은 성능과 확장성으로 이어지는 주요 이점을 제공한다. Key는 Value를 식별하는 고유한 식별자의 역할을 한다. 또한 Value는 해당 Key와 연결된 데이터를 의미한다.

Value는 문자열, 숫자, JSON 객체 등 다양한 형태가 될 수 있으며, 이는 Key-Value Store가 다양한 유형의 데이터를 저장하고 관리할 수 있음을 의미한다.

Key Value Store는 Key를 통하여 직접 데이터에 접근할 수 있기 때문에, 데이터 검색과 저장이 매우 빠르다. 이는 대규모 데이터를 처리할 때 더욱 부각된다.

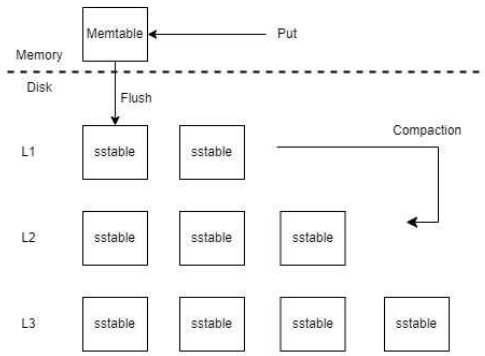
또한 데이터가 Key-Value 쌍으로 저장되기 때문에 수평적 확장이 용이하며, 이는 데이터베이스의 용량을 쉽게 늘릴 수 있음을 의미한다.

마지막으로 Schema-less이기 때문에, 데이터 구조를 미리 정의할 필요가 없다. 이로 인해 다양한 형태의 데이터를 유연하게 저장하고 관리 할 수 있다.

Key Value Store를 구현하는 방법에는 LSM-Tree와 Hash based 방식으로 나뉜다.

Hash based는 비교적 read는 빠르지만 write는 hash collision에 의해 느리다. 이를 사용하는 대표적인 데이터 베이스는 Redis가 있다.

LSM-Tree는 write에 강인한 구조이다. 하지만 비교적read가 느리다는 단점이 있다. 이를 사용하는 대표적인 데이터 베이스는 LevelDB와 RocksDB가 있다.



[그림 1] LSM Tree의 구조

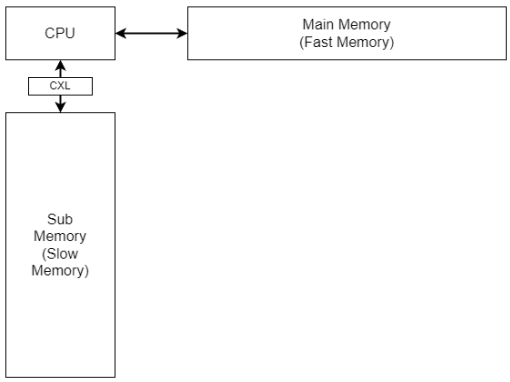
본 연구는 2024년 과학기술정보통신부 및 정보통신기획평가원의 SW중심대학사업의 연구결과로 수행되었음(2022-0-01077)

2.3 LSM-Tree

LSM Tree[1]는 대규모 데이터베이스 시스템에서 쓰기 연산을 최적화하기 위해 개발된 데이터 구조다. 이 구조는 쓰기 작업이 자주 발생하는 환경에서 읽기, 쓰기, 삭제 연산의 성능을 균형 있게 유지하기 위해 설계되었다. LSM-Tree 의 주요 아이디어는 데이터 변경을 즉시 디스크에 반영하는 대신, 먼저 메모리 내의 구조에 기록하고, 일정량이 축적될 때 디스크로 일괄적으로 flush 하는 것이다. 이 과정에서 데이터는 키 기준으로 정렬되어 있어, 디스크 I/O 를 최소화하고 시스템의 전체 성능을 향상시킬 수 있다.

LSM Tree의 구조는 신규 쓰기 작업을 저장하는 MemTable과 데이터를 디스크에 병합하여 저장하는 SSTable로 구성된다. LSM-Tree의 주요 목적은 쓰기 작업을 최적화 하는 것으로, 데이터를 디스크에 저장할 때 MemTable에서 SSTable로의 병합 과정을 통해 이루어진다. 이 과정에서 비효율적인 디스크 접근을 최소화하고, 쓰기 및 읽기 성능을 크게 향상시킬 수 있다.

LSM Tree의 핵심 기능 중 하나는 SSTable 파일의 compaction작업이다. 이 작업을 통해 중복 데이터를 제거하고, 데이터를 더 효율적으로 정렬하여 읽기 성능을 향상 시킨다. 또한, Bloom Filiter를 사용하여 불필요한 디스크 접근을 줄이고, 데이터 조회 시 성능을 개선시킨다.

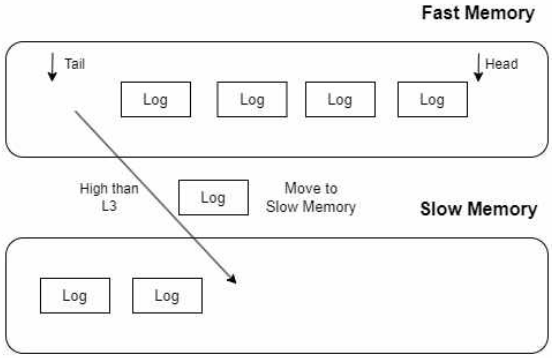


[그림 2] CXL를 통한 Tiered Memory 구조

3. Monolithic구조로 인한 검색시 성능 저하

CPU와 Main Memory가 연결된 구조에서 CXL 인터페이스를 통해 추가적인 Sub Memory를 추가 할 수 있게 되었다. 이를 통해 저장장치(Disk)에 있는 파일시스템에 접근하는 시간보다 빠르게 접근 가능한 추가 저장공간으로써 Sub memory를 사용할 수 있게 되었다. 하지만, 기존 LSM Tree는 Monolithic구조로 구성되어 있기 때문에 Tiered Memory구조에 적합하지 않다. SSTable이 Main memory와 Sub memory에 나누어 저장된다면 Sstable를 검색할 시, Sub memory 또한 검색해야 된다. 이로 인해, SSTable이 Sub Memory에 저장될 시 속도 저하 발생될 수 있다. 이러한 구조를 개선하여 Tiered Memory구조에서도 안정적인 성능을 보여주기 위해 연구를 진행하였다.

4. 연구내용



[그림 3] FastMemory, SlowMemory 내부 구조

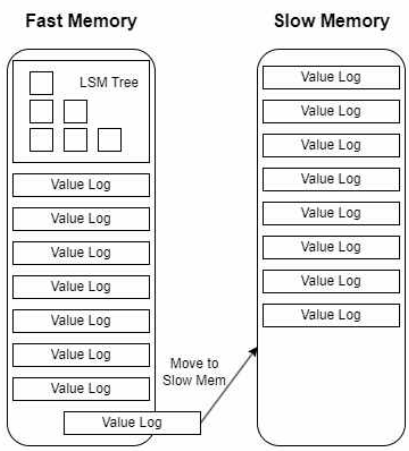
4.1 WiscKey

Fast Memory와 Slow Memory를 사용하는 Tiered Memory 구조에서 LSM Tree를 사용하는 Key-Value Store를 구현하고자 한다. 기존의 방식에서는 각 Level이 Fast Memory와 Slow Memory로 나누어 저장되기 때문에 데이터 처리 시 두 메모리 유형을 모두 검색해야 한다. 특히, Slow Memory는 Fast Memory에 비해 I/O속도가 느리기 때문에 전체 시스템의 성능 저하를 초래할 수 있다. 이 문제를 해결하기 위해 WiscKey[2]방식을 사용하였다.

WiscKey는 LSM Tree 구조를 기반으로 하되, Key와 Value을 분리하여 저장하는 방식을 채택한다. 데이터가 입력될 때, Key와 Value는 분리되어 처리되며, Value는 로그 형식의 파일로 저장된다. 이 파일의 위치 정보(Location)는 Key와 함께 LSM Tree 내에서 정렬되어 저장된다. 이러한 구조는 Value의 크기에 관계없이 LSM Tree의 크기를 줄일 수 있도록 한다.

이를 통해 LSM Tree의 크기가 축소되어 Fast Memory 내에 전체적으로 저장할 수 있다. 반면, Log File은 Fast Memory와 Slow Memory에 나누어 저장되지만, LSM Tree에 Value의 정확한 location이 담겨있기 때문에 이전 방식에서 요구되었던 Slow Memory의 검색 작업이 사라진다.

이러한 구조 덕분에, Tiered Memory 구조에서 LSM Tree 검색 시 Slow Memory의 접근 없이 더 효율적인 I/O 작업을 진행할 수 있다.



[그림 4] Log file Garbage Collection

4.2 Log file Garbage Collection

Log file을 저장할 때, 더 자주 사용하는 Log file은 Fast memory에, 덜 사용되는 Log file은 Slow Memory에 저장하고자 한다. 이때, 나누어 저장된 Log file을 관리하며, Fast Memory에 저장된 자주 사용하지 않는 Log file을 Slow Memory로 이동시키기 위해 Garbage Collection을 사용한다.

Log file Garbage Collection에서는 로그 파일을 지정하는 변수에는 데이터를 추가할 위치인 head와 신뢰할 수 있는 데이터 영역의 시작점을 나타내는 tail 변수가 있다. tail부터 head까지의 데이터는 신뢰할 수 있는 데이터로 간주 된다. Garbage Collection이 실행된다면, tail의 데이터를 head로 이동시킨다. 이 과정 중에 LSM 트리에서 삭제된 데이터를 로그 파일에서도 제거한다. 또한, LSM Tree의 상위 레벨의 데이터는 상대적으로 덜 사용하는 데이터이다. 따라서, 이와 대응되는 Log file을 Slow Memory로 이동시키기 위해 tail에서 head로 데이터를 이동시킬 때 해당 Log file을 Slow Memory로 이동시킨다. 자주 사용하는 데이터는 Fast Memory에 유지하여 빠른 I/O를 가능하게 하고, 상대적으로 사용 빈도가 낮은 파일은 Slow Memory로 이동시킨다.

LSM Tree의 특성상 SSTable 내부의 데이터를 직접 수정할 수 없으므로, 데이터가 업데이트 될 때는 새로운 입력으로 들어오게 되며, 여러 개의 동일한 Key가 있더라도 읽기 과정에서는 가장 최신의 Key를 읽는다. 이러한 특성을 활용하여, 업데이트되는 Key-Value의 쌍은 LSM Tree 의 하위 레벨에 위치할 것이라고 가정하기 때문에 이러한 방법을 사용할 수 있다. 읽기 과정에서는 글로벌 캐시에서 히트한다고 가정하여, 읽기 상황에서도 이 문제를 해결할 수 있다.

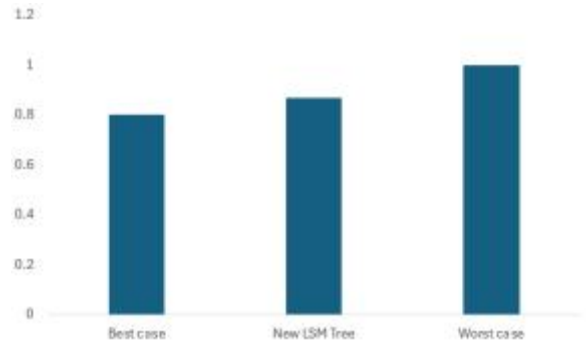
5. 실험

5.1 실험환경

NUMA 시스템에서 remote 메모리의 성능이 local 메모리보다 낮은 특징을 이용하여 Tiered memory구조를 형성하였다. Tiered memory구조에서 기존의 LSM Tree 성능은 모든 Sstable이 Main memory(Fast memory)에 위치할 때 가장 좋은 성능을 보이며, 모든 Sstable이 Sub memory에 위치할 때 가장 나쁜 성능을 보인다.

개선한 LSM Tree는 Tiered Memory구조에서 안정적인 성능이 나옴을 보여주기 위하여 위의 Best Case, Worst Case와 성능을 비교한다. LSM Tree의 Memory Table의 크기가 256 MB인 상태에서, 30개의 스레드가 동시에 100,000번의 input을 진행하여 모든 input이 처리되는 시간을 측정하였다.

5.2 실험 결과



[그림 5] 기존 LSM Tree와 개선된 LSM Tree의 처리 시간 비교

실험 결과 기존 LSM Tree에서 Best case인 경우와 Worst case인 경우는 20%의 성능 차이가 발생하였다. 이에 반해 개선된 LSM Tree는 이 두 성능의 average한 성능을 나타내었다.

6. 결론

본 연구는 LSM Tree의 구조를 개선하여, Tiered Memory구조에서 적합한 LSM Tree를 만들고자 하였다. CPU의 주 메모리를 Fast Memory로, 서브 메모리를 Slow Memory로 가정하였다. Log Garbage Collection 과정에서, Log File을 Fast Memory에서 Slow Memory로 이동시키는 방식을 채택하였다. LSM Tree는 자주 참조되므로 빠른 메모리에 배치할 필요성이 있었고, 이를 위해 WiscKey를 사용하여 LSM Tree의 크기를 축소하였다. 앞서 언급한 실행 결과를 통해, 최악의 경우와 최선의 경우 사이에서 평균적인 성능을 보여주었으며, 이는 Tiered Memory구조에서 기존의 LSM Tree 구조와 비교하여 안정적인 성능을 제공할 것으로 기대된다.

7. 참고문헌

[1] O'Neil, Patrick; Cheng, Edward; Gawlick, Dieter; O'Neil, Elizabeth, "The log-structured merge-tree (LSM-tree)", Acta Informatica. 33, 351-385, (1996-06-01).

[2] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, "WiscKey: Separating Keys from Values in SSD-conscious Storage", (FAST '16), 133-148, (2016-2)